

DEEP NEURAL NETWORKS (AIML ZG565) · COMPANION READER

Linear Neural Networks for Regression

How a single neuron with an identity activation learns to predict continuous values

Session 3 (10 May 2026) · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the formulas; this reader gives you the *why*. Every slide concept is expanded into a full, plain-English explanation, and every slide example is worked out step by step. Colour-coded boxes guide you: **blue** = intuition; **amber** = everyday picture; **green** = worked example; **red** = watch out; **purple** = key takeaway.

1 What this session covers

Session 3 instantiates the four-ingredient DL framework on the simplest possible model: *linear regression*. One neuron. One loss. One learning algorithm. The goal is to make the framework concrete before adding hidden layers, non-linear activations, and classification. Every concept here — the design matrix, the squared-error loss, the gradient derivation, batch gradient descent, and the three evaluation metrics — reappears unchanged in deep networks; only the model grows.

2 What is regression?

Regression is a supervised learning task that predicts a **continuous-valued output** from a vector of input features. Formally: the input is a feature vector $\mathbf{x} \in \mathbb{R}^d$ and the output is a real number $y \in \mathbb{R}$. The goal is to learn a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that $y \approx f(\mathbf{x})$.

The intuition

Regression answers “how much?” or “how many?” — a number on a continuous scale, not a named category. Contrast with classification: classification answers “which one?” (spam vs. not-spam); regression answers “how expensive?” or “how many degrees?”.

★ Everyday picture

A doctor measuring a patient’s blood pressure gets a number like 120 mmHg, not a category. Predicting that number from age, BMI, and diet is regression — because the target lives on a continuous scale.

Slide examples: house price prediction (input = size, location, rooms; output = price in dollars); temperature forecasting (input = historical weather, season, humidity; output = temperature in degrees); stock price prediction (input = historical prices, volume, indicators; output = future price). All outputs are real numbers — regression.

Where this shows up in ML. Regression is half of supervised learning. Every deep network that

predicts temperatures, prices, or pixel values at the end uses a regression head (a linear output layer), so this session's math applies to the final layer of almost every model you will ever train.

✓ Key takeaway

Regression = supervised learning with a continuous real-valued output. The model learns f such that $y \approx f(\mathbf{x})$ for any input $\mathbf{x}$.

3 Linear regression: the model

Linear regression assumes the output is a *linear combination* of the input features. With d features $x_1, \dots, x_d$:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d, \quad (1)$$

where w_0 is the **bias** (a constant offset) and $w_1, \dots, w_d$ are the **weights** (slopes on each feature).

🧠 The intuition

Each weight w_i says: “for every one-unit increase in feature x_i , the prediction changes by w_i , regardless of the other features.” That independence of effects is what makes the model *linear*.

Slide examples of linear regression:

- **Salary prediction:** Salary = $w_0 + w_1 \times \text{Years of experience}$. One weight captures the salary rise per extra year.
- **Crop yield:** Yield = $w_0 + w_1 \times \text{Rainfall} + w_2 \times \text{Fertilizer}$. Two weights, one per input.
- **Energy consumption:** Energy = $w_0 + w_1T + w_2H + w_3D$. Three features — temperature T , hour H , day-of-week D .

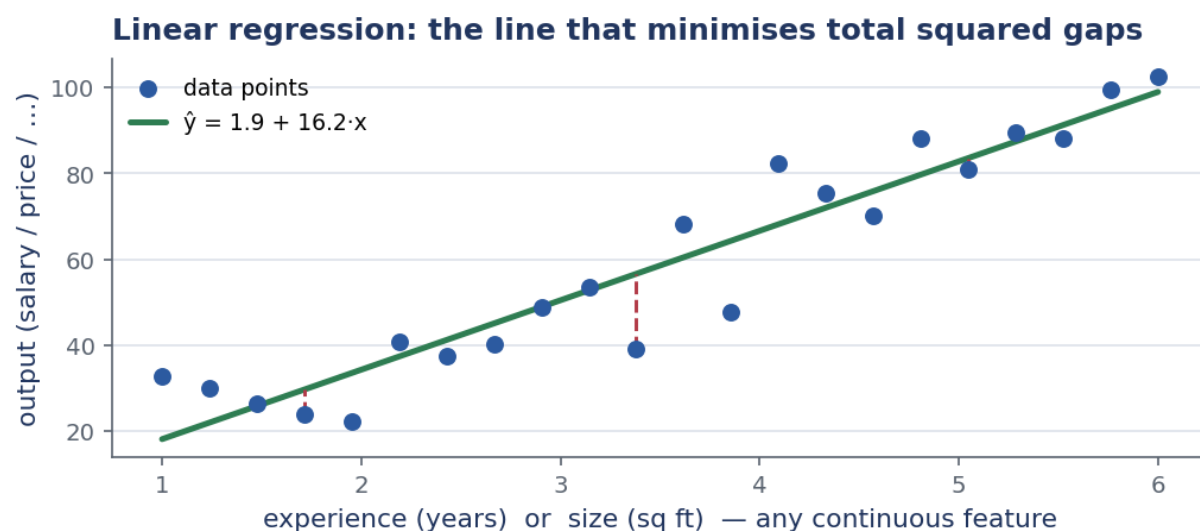


Figure 1: How to read it: each dot is one data point; the green line is the fitted linear model. The red dashed gaps between a point and the line are the residuals — the errors the model makes. Training minimises the sum of squared residuals.

Compact vector notation. Prepend a 1 to $\mathbf{x}$ to absorb the bias into the weight vector:

$$\hat{y} = \mathbf{w}^\top \mathbf{x}, \quad \mathbf{w} = [w_0, w_1, \dots, w_d]^\top \in \mathbb{R}^{d+1}, \quad \mathbf{x} = [1, x_1, \dots, x_d]^\top \in \mathbb{R}^{d+1}. \quad (2)$$

For all N training examples at once, stack the inputs into a **design matrix** $X \in \mathbb{R}^{N \times (d+1)}$ (each row is one augmented example) and all predictions become:

$$\hat{\mathbf{y}} = X\mathbf{w}. \quad (3)$$

Where this shows up in ML. The vectorised form $\hat{\mathbf{y}} = X\mathbf{w}$ is identical to the computation in a linear layer with no activation — the building block of every deep network’s final regression head.

⚠ Watch out

Linear regression captures only linear relationships. If the true mapping is curved (e.g. house price rising non-linearly with size), the linear model will systematically under- or over-predict in certain regions. Adding hidden layers with non-linear activations is the deep-learning fix.

✓ Key takeaway

Linear regression: $\hat{y} = \mathbf{w}^\top \mathbf{x}$ (bias absorbed into $\mathbf{w}$ and a prepended 1). For N examples: $\hat{\mathbf{y}} = X\mathbf{w}$.

4 The model as a single neuron

The slides frame linear regression as a *single artificial neuron* with the **identity activation function** $f(z) = z$. The neuron takes all d features plus a bias input, forms the weighted sum z , and passes it through f :

$$z = w_0 + w_1x_1 + \dots + w_dx_d, \quad (4)$$

$$\hat{y} = f(z) = z. \quad (5)$$

Linear regression as a single neuron with identity activation $f(z) = z$

$$\hat{y} = f(w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d) = w_0 + w_1x_1 + \dots + w_dx_d \quad (\text{identity activation})$$

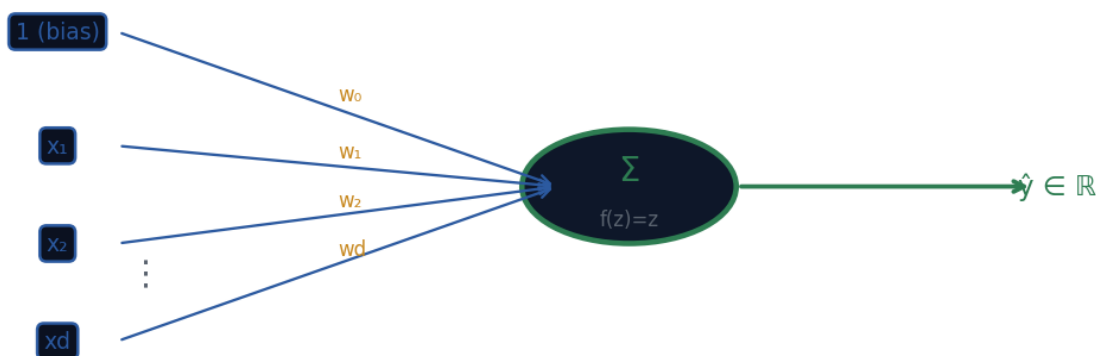


Figure 2: How to read it: arrows represent weights feeding into the summation node Σ . The activation $f(z) = z$ (identity) leaves the sum unchanged, so the output is continuous and unbounded — the right shape for predicting real-valued targets.

🧠 The intuition

The identity activation does nothing to z — it just passes it through. That is deliberate: we need the output to be any real number (not clamped to $[0, 1]$ or $\{0, 1\}$). For regression, “no squashing” is the correct activation.

Why the identity activation for regression?

- Output is continuous and unbounded: matches real-valued targets (prices, temperatures).
- Differentiable everywhere with gradient 1: gradient flows unchanged through the neuron — crucial for gradient descent.
- No information loss: the prediction is exactly the weighted sum, losing nothing.

Where this shows up in ML. Every deep regression network ends with a single neuron with the identity activation. Even GPT, when used to predict a continuous score, has this at the top.

✓ Key takeaway

Linear regression = a single neuron with identity activation $f(z) = z$. Identity preserves the gradient exactly ($f'(z) = 1$) and allows any real output.

5 The objective: squared error and MSE loss

To train the model, we need a score that is low when predictions are close to targets and high when they are far. The **squared error** for one example is $(y - \hat{y})^2$. Averaging over all N training examples gives the **mean squared error (MSE)** loss:

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2 = \frac{1}{N} \|\mathbf{y} - X\mathbf{w}\|^2. \quad (6)$$

Here $y^{(i)}$ is the i -th true label and $\hat{y}^{(i)} = \mathbf{w}^\top \mathbf{x}^{(i)}$ is the i -th prediction.

★ Everyday picture

You guess 20°C for today's temperature and the truth is 25°C. Your squared error is $(25 - 20)^2 = 25$. Squaring does two things: it makes the sign of the error irrelevant (overshot and undershot both count), and it *amplifies big misses*: an error of 10 costs 100, ten times an error of 5 that costs only 25.

Why squared error?

- **Differentiable** everywhere: a smooth surface that gradient descent can navigate.
- **Convex** for linear models: one global minimum, no local traps. (See the bowl in the figure below.)
- **Penalises large errors quadratically**: forces the model to pay close attention to the worst misses.
- **Statistical interpretation**: MSE is the **maximum likelihood estimate** under the assumption that the prediction errors are Gaussian (normally distributed) noise. Minimising MSE equals maximising the likelihood of the data under that noise model.

Where this shows up in ML. MSE is the default regression loss in PyTorch (`nn.MSELoss`), TensorFlow, and scikit-learn. The Gaussian noise interpretation is also why regression outputs are often interpreted as the mean of a Gaussian predictive distribution.

✗ Watch out

MSE is sensitive to outliers: a single large error contributes a huge squared term. In datasets with noisy outliers, **mean absolute error (MAE)** — using $|y - \hat{y}|$ instead of $(y - \hat{y})^2$ — is more robust.

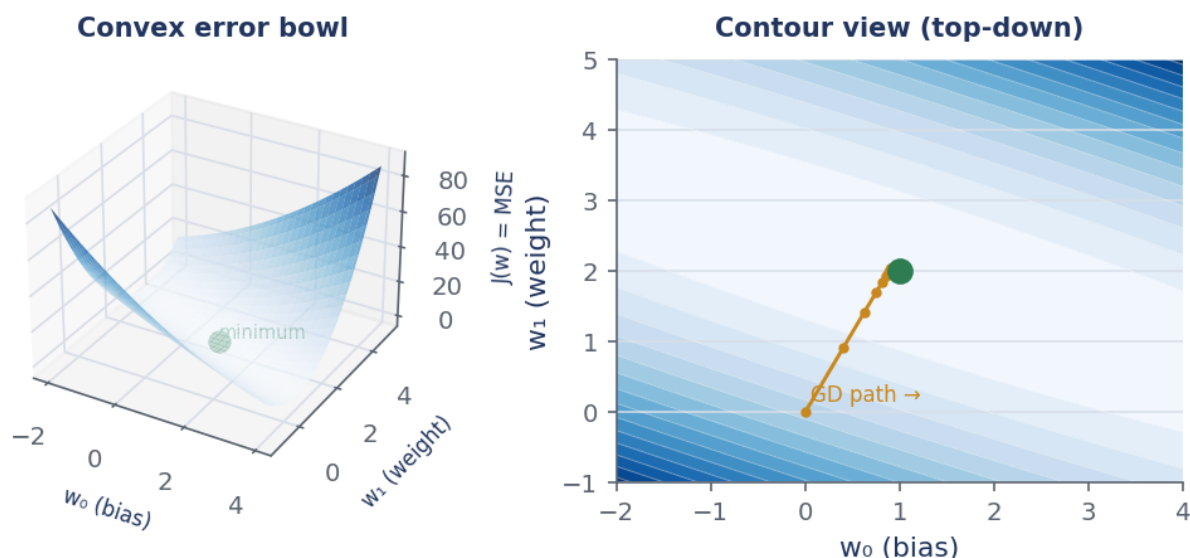


Figure 3: How to read it: left panel is the 3D bowl — MSE as a function of two parameters w_0 (bias) and w_1 (weight). The green dot is the global minimum. Right panel is the contour view (top-down slice); the amber path is a gradient descent trajectory converging toward the minimum.

MAE is less smooth (not differentiable at 0), but usable with subgradients.

✓ Key takeaway

$J(\mathbf{w}) = \frac{1}{N} \sum (y^{(i)} - \hat{y}^{(i)})^2$. Convex for linear models (one global minimum); amplifies large errors; equals the Gaussian negative log-likelihood.

6 The learning algorithm: batch gradient descent

The learning algorithm searches for the weights $\mathbf{w}^*$ that minimise $J(\mathbf{w})$. For linear regression (convex loss) the global minimum exists and gradient descent provably finds it.

The gradient. The partial derivative of J with respect to $\mathbf{w}$ gives the direction of steepest ascent. Moving in the *opposite* direction descends the error bowl. The full derivation:

$$J(\mathbf{w}) = \frac{1}{N} \|\mathbf{y} - X\mathbf{w}\|^2, \quad (7)$$

$$\nabla_{\mathbf{w}} J = \frac{2}{N} X^\top (X\mathbf{w} - \mathbf{y}) = \frac{2}{N} X^\top (\hat{\mathbf{y}} - \mathbf{y}). \quad (8)$$

Each element says: *how much does the loss change if I nudge this weight by a tiny amount?* A positive gradient means “nudging w_i up would increase the loss,” so we nudge it *down*.

The update rule. At each iteration t :

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla J(\mathbf{w}^{(t)}), \quad (9)$$

where $\eta > 0$ is the **learning rate** (step size). The batch gradient descent algorithm uses *all* N training examples to compute the gradient at each step (“batch” = whole training set):

1. Initialise $\mathbf{w}^{(0)}$ (typically $\mathbf{0}$).

2. Repeat until convergence:

- (a) Compute predictions: $\hat{\mathbf{y}} = X\mathbf{w}$.
- (b) Compute error: $\mathbf{e} = \hat{\mathbf{y}} - \mathbf{y}$.
- (c) Compute gradient: $\nabla J = \frac{2}{N}X^\top \mathbf{e}$.
- (d) Update weights: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$.

★ Everyday picture

A pastry chef adjusts a recipe. She bakes a batch (all the cakes), tastes each one, averages the feedback, and makes one big adjustment to the recipe. That's batch gradient descent. A chef who adjusts after each individual cake is *stochastic* gradient descent (SGD) — faster but noisier.

✍ Worked example: two steps of batch gradient descent on the slide's numerical example

Setup (from slides): three training points, one feature each. $x^{(1)} = 1$, $y^{(1)} = 2$; $x^{(2)} = 2$, $y^{(2)} = 4$; $x^{(3)} = 3$, $y^{(3)} = 5$. Design matrix (prepend 1 for bias):

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix}, \quad \mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \eta = 0.1. \quad (10)$$

Step 1. Initial predictions. $\hat{\mathbf{y}} = X\mathbf{w}^{(0)} = [0, 0, 0]^\top$.

Step 2. Initial errors. $\mathbf{e} = \hat{\mathbf{y}} - \mathbf{y} = [-2, -4, -5]^\top$.

Step 3. Initial loss.

$$J^{(0)} = \frac{1}{3}(4 + 16 + 25) = \frac{45}{3} = \mathbf{15.0}. \quad (11)$$

Step 4. Compute gradient.

$$\nabla J = \frac{2}{3}X^\top \mathbf{e} = \frac{2}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix} = \frac{2}{3} \begin{bmatrix} -11 \\ -23 \end{bmatrix} = \begin{bmatrix} -7.333 \\ -15.333 \end{bmatrix}. \quad (12)$$

Step 5. First weight update.

$$\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - 0.1 \times \begin{bmatrix} -7.333 \\ -15.333 \end{bmatrix} = \begin{bmatrix} 0.733 \\ 1.533 \end{bmatrix}. \quad (13)$$

Step 6. New predictions and loss. $\hat{\mathbf{y}}^{(1)} = X\mathbf{w}^{(1)} = [2.267, 3.800, 5.333]^\top$. $\mathbf{e}^{(1)} = [0.267, -0.200, 0.333]^\top$.

$$J^{(1)} = \frac{1}{3}(0.071 + 0.040 + 0.111) = \frac{0.222}{3} \approx \mathbf{0.074}. \quad (14)$$

One step dropped the loss from **15.0** to **0.074** — a 99.5% reduction!

Step 7. Sense-check. The weights $w_0 = 0.733 \approx 1$ and $w_1 = 1.533 \approx 1.5$ are converging toward a good fit. The large gradient norms in step 4 caused a big first jump; subsequent steps will be smaller as the error shrinks.

Where this shows up in ML. Batch gradient descent is the cleanest theoretical form. In practice, large

Gradient descent on MSE: loss falls each step as weights improve

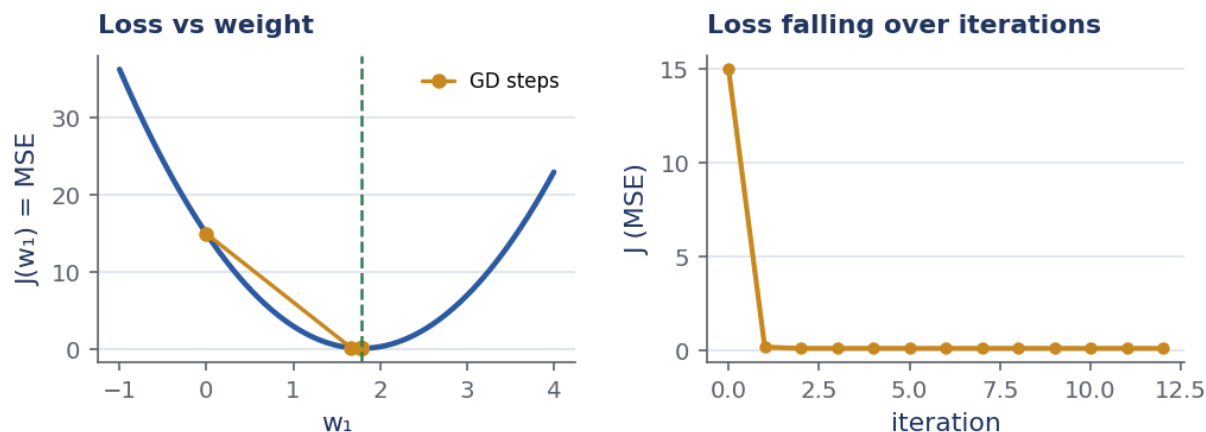


Figure 4: How to read it: left panel — the amber dots trace the gradient descent path along the loss curve; the loss (vertical axis) falls with each step. Right panel — the same loss plotted against iteration number, showing the characteristic fast early drop.

datasets use **mini-batch stochastic gradient descent** (SGD): randomly sample a small batch (e.g. 32 or 256 examples) at each step, compute the gradient on that batch alone, and update. It's noisier but much faster per update and can escape shallow local minima in deep networks.

⚠ Watch out

Choosing η (the learning rate) matters enormously. Too large: the update overshoots the minimum, J bounces or diverges. Too small: convergence is agonisingly slow. Standard practice: start with $\eta \approx 10^{-3}$ and use a learning rate scheduler that reduces η over time.

✓ Key takeaway

Batch GD: compute $\nabla J = \frac{2}{N} X^\top (\hat{\mathbf{y}} - \mathbf{y})$ on all examples; update $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$; repeat. Convexity guarantees convergence to the global minimum for linear regression.

7 Model evaluation

A model that memorises the training set but fails on new data is useless. Evaluation always uses *held-out* data — examples the model has never seen — to estimate real-world performance.

Training error is the loss on the data used for training. **Test error** (or generalisation error) is the loss on fresh examples. Good models have both errors low; a big gap (low training, high test) signals **overfitting**.

Three standard regression metrics:

1. **Mean Squared Error (MSE):** $\text{MSE} = \frac{1}{N} \sum (y^{(i)} - \hat{y}^{(i)})^2$. Same units as y^2 ; sensitive to outliers.
2. **Mean Absolute Error (MAE):** $\text{MAE} = \frac{1}{N} \sum |y^{(i)} - \hat{y}^{(i)}|$. Same units as y ; more robust to outliers.

Evaluation: train error $\leq$ test error (test is the real measure)

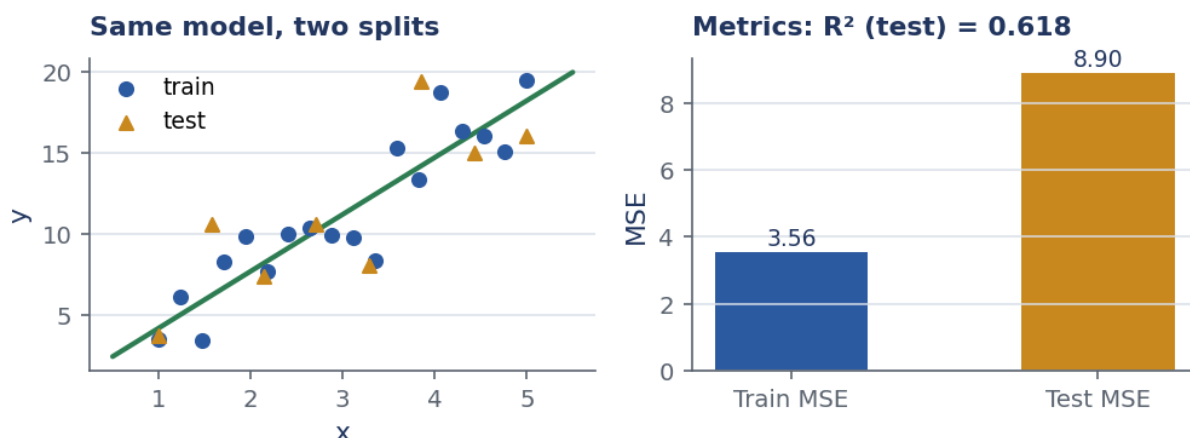


Figure 5: How to read it: left panel shows the same fitted line evaluated on the training set (blue circles) and the test set (amber triangles). Right panel compares the two MSE values — the test error is typically higher. A model is only as good as its test error.

3. R^2 (coefficient of determination):

$$R^2 = 1 - \frac{\sum (y^{(i)} - \hat{y}^{(i)})^2}{\sum (y^{(i)} - \bar{y})^2}. \quad (15)$$

Here $\bar{y}$ is the mean of the true labels. $R^2 = 1$ means perfect predictions; $R^2 = 0$ means the model does no better than always predicting $\bar{y}$; $R^2 < 0$ means the model is worse than that naive baseline.

Worked example: computing MSE, MAE, and R^2 on three test points

True values: $y = [3, 5, 4]$. Predictions: $\hat{y} = [2.5, 5.5, 3.5]$. $\bar{y} = 4$.

Step 1. Errors: $e = y - \hat{y} = [0.5, -0.5, 0.5]$.

Step 2. MSE: $\frac{1}{3}(0.25 + 0.25 + 0.25) = \frac{0.75}{3} = \mathbf{0.25}$.

Step 3. MAE: $\frac{1}{3}(0.5 + 0.5 + 0.5) = \frac{1.5}{3} = \mathbf{0.5}$.

Step 4. R^2 numerator: $\sum e^2 = 0.75$.

Step 5. R^2 denominator: $\sum (y^{(i)} - \bar{y})^2 = (-1)^2 + 1^2 + 0^2 = 2$.

Step 6. R^2 : $1 - 0.75/2 = 1 - 0.375 = \mathbf{0.625}$. The model explains 62.5% of the variance in y .

Step 7. Sense-check. All three metrics are small (good), and $R^2 > 0$ confirms the model beats the trivial “always predict $\bar{y} = 4$ ” baseline.

Where this shows up in ML. `sklearn.metrics.mean_squared_error`, `mean_absolute_error`, and `r2_score` compute exactly these. R^2 is especially useful because it is scale-free: $R^2 = 0.9$ means the same quality whether prices are in rupees or dollars.

Watch out

R^2 can be misleading when evaluating on very small test sets or when the target distribution changes between train and test splits. Always report both a metric value and the number of test examples.

✓ Key takeaway

MSE (squared errors), MAE (absolute errors), R^2 (fraction of variance explained). Use MAE when outliers are present; use R^2 for scale-independent comparisons.

8 Implementation tips

The slides give practical advice for coding linear regression in Python. The key points:

- **Feature scaling.** Standardise each feature to zero mean and unit variance before training: $x_j \leftarrow (x_j - \mu_j)/\sigma_j$. Without this, features on different scales (e.g. salary in thousands vs. age in years) cause very different gradient magnitudes, slowing convergence or causing numerical overflow.
- **Initialise weights to zero** (or small random values). For linear regression the choice rarely matters (convex loss, no symmetry breaking needed), but zero init is clean and reproducible.
- **Monitor the loss.** Print or plot J every 100 iterations. If it is falling smoothly, training is healthy. If it is flat (learning rate too small) or rising/oscillating (too large), adjust η .
- **Vectorise.** Use NumPy matrix operations ($X\mathbf{w}$, $X^\top \mathbf{e}$) instead of Python loops. One matrix multiply is hundreds of times faster than looping over examples.
- **Debugging checklist:** shapes first (confirm X is $N \times (d+1)$ and $\mathbf{y}$ is $N \times 1$); then check J is falling; then confirm predictions look reasonable on a few training examples.

✗ Watch out

Apply feature scaling using *training* statistics only. Compute μ_j and σ_j on the training set and use those same values to scale the test set. Scaling the test set using test statistics would be data leakage.

✓ Key takeaway

Scale features, vectorise with NumPy, monitor the loss, and always check array shapes. Data leakage through incorrect test-set scaling is the most common implementation mistake.

9 Recap and what comes next

One paragraph to carry away. Regression predicts a continuous y from input $\mathbf{x}$. Linear regression models this as a single neuron with identity activation $f(z) = z$, yielding predictions $\hat{\mathbf{y}} = X\mathbf{w}$. Training minimises the MSE loss $J(\mathbf{w}) = \frac{1}{N} \|\mathbf{y} - X\mathbf{w}\|^2$ using batch gradient descent: compute gradient $\nabla J = \frac{2}{N} X^\top (\hat{\mathbf{y}} - \mathbf{y})$, update $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$, repeat. For linear models, this is convex and converges to the global minimum. Evaluate with MSE, MAE, and R^2 on held-out test data.

✓ Key takeaway

Linear regression = identity neuron + MSE loss + batch gradient descent. The *same* pipeline — forward pass, loss, gradient, update — scales to any deep network; only the model definition changes.

Further reading. Zhang et al. *Dive into Deep Learning*, Chapter 3 (linear regression); Goodfellow et al., Chapter 5 (machine learning basics); Python notebook linked in the slides.

Bridge to Session 4. The output here is a real number. What if we want a probability — or a class label? We swap the identity activation for a sigmoid (binary) or a softmax (multi-class), swap MSE for cross-entropy, and we have *logistic regression* and *linear classification*. That is Session 4.